

Squad Orchestrator

Domínios, entidades e contratos da orquestração de squads agênticas sobre kilo serve

O sistema recebe uma tarefa de código, isto é, um repositório, uma branch e um pedido, e a entrega a uma squad de agentes que trabalha isolada numa sandbox própria. Três processos participam: o navegador, que só conversa com o orquestrador; o orquestrador, que provisiona containers, assina o stream de cada sandbox e republica eventos já classificados; e o kilo serve, que roda dentro de cada container, executa a squad e fala com o provedor de LLM.

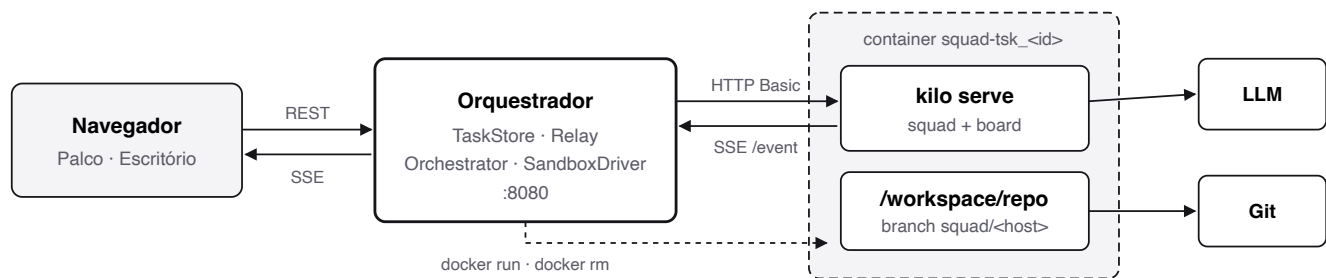


Figura 1 — Contexto. O navegador nunca vê a URL nem a senha do kilo serve; a porta do container é publicada apenas no loopback do host.

1. Mapa de domínios

Sete domínios dividem o sistema. Os quatro primeiros vivem no orquestrador, o quinto pertence ao Kilo e é apenas observado, e os dois últimos ficam na interface.

DOMÍNIO	RESPONSABILIDADE	ENTIDADES	ONDE VIVE
Tarefas	Aceitar pedidos, manter o ciclo de vida e responder à API	Task, CreateTaskInput, TaskStatus	store.ts, routes.ts
Sandbox	Criar, vigiar e destruir o container de cada tarefa	SandboxHandle, SandboxDriver	sandbox.ts
Orquestração	Fila de provisionamento, sessão raiz, prompts, abort e limpeza por ociosidade	Orchestrator	orchestrator.ts
Eventos	Assinar o SSE do Kilo, classificar, gravar em log e retransmitir com replay	KiloEvent, UiEvent, UiEventKind	relay.ts
Squad e coordenação	Agentes, delegações e board do Swarm (executado pelo Kilo)	SquadMember, SessionBoard, BoardMessage	sandbox/squad/
Governança	Política de ferramentas e aprovação humana de pedidos	PermissionRequest, PermissionReply	kilo.jsonc, API
Apresentação	Reconstruir o estado a partir do stream e desenhar Palco e Escritório	TaskState, AgentNode, SquadModel	web/src

2. Modelo de entidades

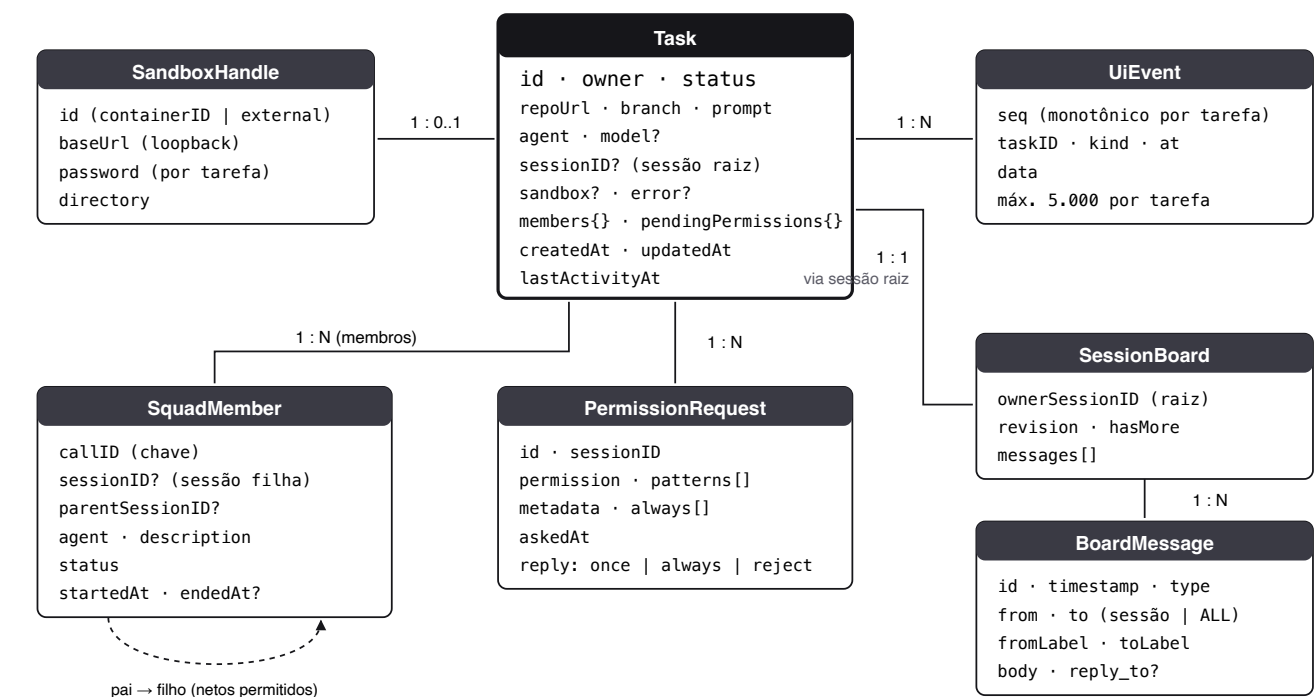


Figura 2 — Entidades do orquestrador e do Kilo. Task é a raiz do agregado; Board e mensagens pertencem ao Kilo e são lidos por snapshot.

2.1 Task

Raiz do agregado. Cada tarefa tem uma sandbox, uma sessão raiz no Kilo e um log próprio de eventos. O dono é fixo (poc-user) enquanto não houver autenticação real.

CAMPO	TIPO	REGRA
id	string	tsk_ seguido de 20 caracteres hexadecimais
owner	string	Derivado do token; filtra a listagem
repoUrl · branch	string	Branch vazia significa a branch padrão do repositório
prompt	string	Primeiro turno da sessão raiz; os 80 primeiros caracteres viram o título da sessão
agent	string	Agente primário da sessão; padrão squad-lead
model	string?	Formato provider/model; na ausência vale DEFAULT_MODEL
status	TaskStatus	Ver ciclo de vida (seção 3)
sandbox	SandboxHandle?	Preenchido após o provisionamento; nunca sai da API pública
sessionID	string?	Sessão raiz; também é a dona do board
members	Record<callID, SquadMember>	Uma entrada por delegação feita com a tool task
pendingPermissions	Record<id, PermissionRequest>	Somente os pedidos ainda sem resposta
lastActivityAt	epoch ms	Base do descarte por ociosidade

2.2 SandboxHandle

CAMPO	REGRA
id	ID do container Docker, ou external quando o driver aponta para um kilo serve já em execução
baseUrl	http://127.0.0.1:<porta aleatória>, publicada só no loopback
password	Gerada por tarefa; autenticação Basic com usuário kilo
directory	/workspace/repo, repassado como ?directory= em todas as rotas do Kilo

2.3 SquadMember

Representa uma delegação, e não um agente. Cada chamada da tool task cria uma sessão filha, com chave no callID. Quando o líder delega de novo ao mesmo agente, por exemplo depois de um VETO, surge outro SquadMember com outra sessão filha.

CAMPO	REGRA
callID	ID da tool call no Kilo; chave primária
sessionID	Sessão filha, lida de state.metadata.sessionId
parentSessionID	Sessão que delegou; permite netos
agent · description	Tipo do subagente e título da delegação
status	pending → running → completed error

2.4 PermissionRequest

Aberto pelo Kilo sempre que uma ferramenta cai em ask. Enquanto houver algum pendente, a tarefa fica em waiting_permission. A resposta vai para POST /permission/:id/reply com once, always ou reject, mais uma mensagem opcional.

2.5 SessionBoard e BoardMessage

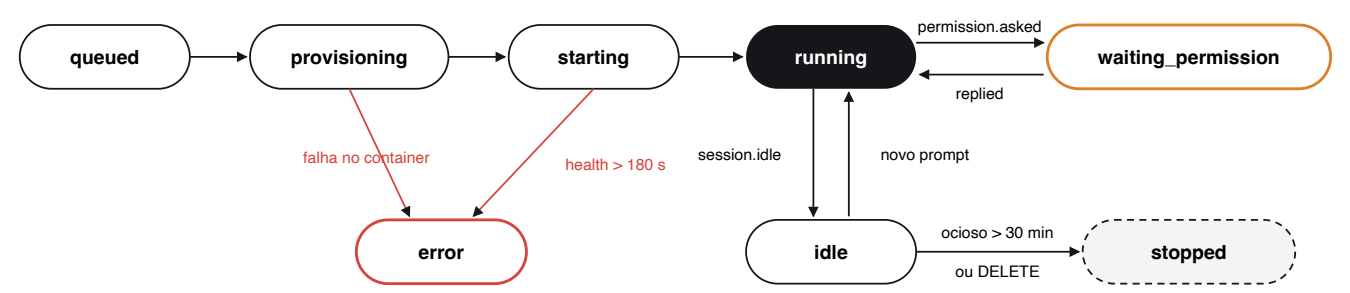
O board do Swarm pertence à sessão raiz e tem revisão incremental. As mensagens circulam entre sessões ou são enviadas para ALL, e podem responder a outra pelo campo reply_to. O reset exige a revisão corrente.

TIPO	SIGNIFICADO	QUEM EMITE
INFO	Aviso ou plano, sem pedir resposta	squad-lead
ASK	Pergunta bloqueante dirigida a alguém	developer, qualquer membro
RESULT	Entrega com resumo e forma de verificar	todos
HOLD	Falha que pede parada, com a saída relevante	qa
VETO	Problema bloqueante com arquivo, linha e correção	reviewer

2.6 UiEvent

Unidade do stream para o navegador. O seq cresce por tarefa e serve de id SSE. Uma reconexão com Last-Event-ID ou ?after= recebe tudo o que ficou para trás, dentro do limite de 5.000 eventos guardados por tarefa.

3. Ciclo de vida da tarefa



Qualquer estado vivo chega a stopped por DELETE /tasks/:id. Retentativas do LLM (session.status = retry) não mudam o status da tarefa.

Figura 3 — Estados de TaskStatus e seus gatilhos.

ETAPA	O QUE ACONTECE	LIMITE
queued	Tarefa aceita (HTTP 202) e posta na fila de provisionamento	4 provisionamentos simultâneos
provisioning	docker run com rótulos squad-orchestrator=1 e squad-task=<id>	2 CPUs, 4 GB, 1.024 processos
starting	Clone do repositório, subida do kilo serve e espera por /global/health	180 s; container morto encerra na hora
running	SSE assinado, sessão raiz criada, prompt enviado	SSE precisa abrir em 15 s
idle	Turno concluído; a sandbox continua viva para follow-ups	30 min sem atividade
stopped	Container destruído e stream desligado	Final

4. A squad

A squad é definida fora do repositório, em sandbox/squad/, e carregada pelo Kilo via KILO_CONFIG_DIR. Se o repositório trazer .kilo/agents/*.md, esses arquivos também valem e podem sobrescrever agentes de mesmo nome.

AGENTE	MODOS	PAPEL	EDITA?	BOARD	COR
squad-lead	primary	Planeja, delega com task, responde ASK, trata HOLD e VETO, faz o commit	sim	INFO do plano, RESULT final	#e6e6e6
architect	subagent	Desenho: arquivos, contratos, riscos, ordem	não	RESULT para main	#8052ff
developer	subagent	Implementa a partir do desenho e roda testes e lint	sim	ASK se bloqueado, RESULT	#63a1ff
qa	subagent	Descobre como o projeto testa, escreve e roda testes	sim	RESULT ou HOLD	#59d499
reviewer	subagent	Revisa o git diff: correteude, bordas, segurança, estilo	não	VETO ou RESULT	#f2a0ff

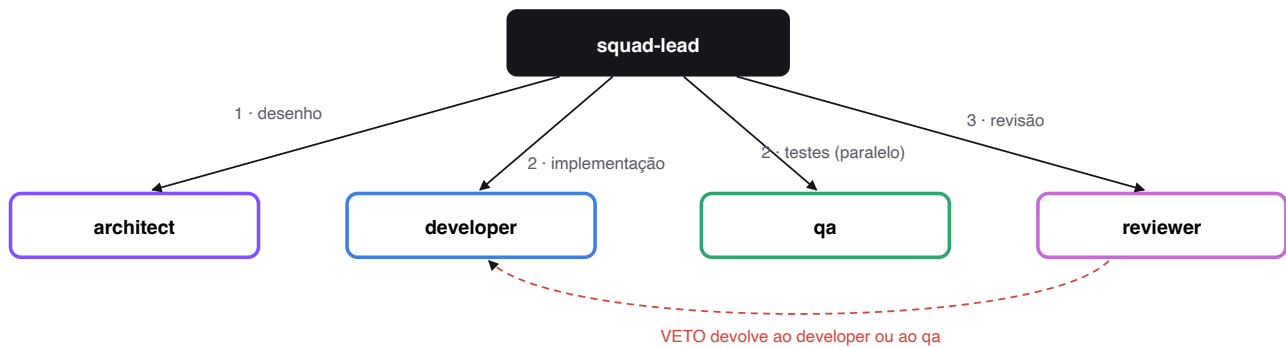


Figura 4 — Delegação padrão do squad-lead. Toda comunicação lateral passa pelo board da sessão raiz.

5. Governança de ferramentas

A política fica em `sandbox/squad/kilo.jsonc`. O que estiver em `ask` vira um `PermissionRequest` e aparece na interface para aprovação humana.

allow

```

read · edit · glob · grep · list · task ·
todowrite

bash: git status · git diff · git log ·
git add · git commit · ls · cat · npm test
· npm run
  
```

ask e deny

```

ask: webfetch · qualquer outro bash · git
push

deny: external_directory

desligado: sandbox bubblewrap
  
```

O isolamento está no container de cada tarefa, com o perfil padrão do Docker.

`SANDBOX_NATIVE_ISOLATION=false` mantém fora os afrouxamentos `seccomp`, `apparmor` e `systempaths`.

Chaves de provedor entram no container apenas pelas variáveis listadas em `SANDBOX_ENV_PASSTHROUGH`; o token do GitHub vira `credential helper` e não aparece na URL do remoto.

6. Vocabulário de eventos

KIND	ORIGEM NO KILO	CONSUMIDOR NA INTERFACE
<code>task.status</code>	orquestrador	Cabeçalho e estado geral
<code>message · message.delta</code>	<code>message.updated</code> , <code>message.part.updated</code> (texto), <code>message.part.delta</code>	Conversa e atividade atual de cada agente
<code>squad.member</code>	tool <code>task</code> , com a sessão filha em <code>state.metadata.sessionId</code>	Nós do grafo e personagens do escritório
<code>board.activity · board.snapshot</code>	tools <code>board_post</code> e <code>board_read</code> , mais <code>GET .../board</code>	Board, balões e post-its
<code>permission.asked · replied</code>	<code>permission.*</code>	Alerta e card de aprovação
<code>tool</code>	demaís tool calls	Linha do tempo e ícone de ação
<code>session.status</code>	<code>busy</code> , <code>idle</code> , <code>retry</code> (com tentativa e mensagem)	Estado dos nós e alerta de retentativa

<code>session.diff · error</code>	<code>session.*</code>	Gaveta de diff e erros
<code>raw</code>	todo o resto, incluindo <code>session.bound</code> , <code>sandbox.ready</code> , <code>sandbox.reaped</code>	Depuração; <code>session.bound</code> fixa a sessão raiz

Cada `board_post` ou `board_read` concluído dispara uma atualização do snapshot do board, limitada a uma a cada 400 ms. Enquanto a sessão estiver ocupada, um polling de 15 s cobre o que escapar.

7. Modelo da apresentação

A interface não guarda estado próprio: reconstrói tudo a partir do snapshot e do stream. Para desenhar, agrupa as delegações por agente.

ENTIDADE	COMPOSIÇÃO
TaskState	task, mensagens e partes, membros, permissões, board e revisão, atividade por sessão, estado por sessão, contagem do diff, erros
AgentNode	Um por agente. O id é a sessão raiz para o líder e <code>agent:<nome></code> para os demais. Guarda todas as rodadas (<code>members</code>) e sessões do agente
SquadModel	Nós, índice por id e <code>resolve()</code> , que converte uma sessão, <code>main</code> ou um nome de agente no nó correspondente
Estado do nó	Esperando você, depois trabalhando, depois o estado da rodada mais recente

Palco

Grafo no centro, com pulsos de delegação, entrega e board, e um letreiro com os últimos acontecimentos. Um clique no nó abre o painel do agente. O rodapé traz a última fala do líder e o campo de follow-up. No topo aparece um único alerta por vez: permissão, depois ASK, HOLD ou VETO em aberto, depois erro.

Escritório

Canvas em pixel art, com uma baia por agente, sala do líder, quadro de post-its e máquina de café. Cada personagem tem uma fila de ações; quando a fila cresce, as caminhadas são puladas e ficam só os balões.

8. Contrato da API

Autenticação por Authorization: Bearer <API_TOKEN> ou ?token= para EventSource. Apenas /health é público.

MÉTODO	ROTA	EFEITO
POST	<code>/tasks</code>	Cria a tarefa (<code>repoUrl</code> , <code>branch?</code> , <code>prompt</code> , <code>agent?</code> , <code>model?</code>) e responde 202
GET	<code>/tasks · /tasks/:id</code>	Lista ou detalha, sem expor a sandbox
GET	<code>/tasks/:id/events</code>	SSE com replay por Last-Event-ID ou ?after=
POST	<code>/tasks/:id/prompt</code>	Follow-up na mesma sessão

POST	/tasks/:id/permissions/:pid	Responde com once, always ou reject
GET · POST	/tasks/:id/board · /board/reset	Snapshot paginado e reset pela revisão
GET	/tasks/:id/diff	Diff da sessão
POST	/tasks/:id/abort	Interrompe o turno em andamento
DELETE	/tasks/:id	Destrói a sandbox

9. Parâmetros operacionais

VARIÁVEL	VALOR PADRÃO	EFEITO
SANDBOX_DRIVER	docker	external aponta para um kilo serve já em execução
SANDBOX_IMAGE	squad-sandbox:dev	node 22, kilo CLI, git, ripgrep, bubblewrap
SANDBOX_CPUS · SANDBOX_MEMORY	2 · 4g	Limites por container
SANDBOX_NETWORK	rede padrão	Rede Docker onde o container é criado
SANDBOX_NATIVE_ISOLATION	false	t rue religa o bubblewrap e os afrouxamentos do container
SQUAD_LEAD_AGENT	squad-lead	Agente primário das sessões
DEFAULT_MODEL	—	Modelo usado quando a tarefa não informa
IDLE_TTL_MINUTES	30	Tempo até o descarte de uma sandbox ociosa

O armazenamento é em memória, com tarefas e log de eventos. A interface do TaskStore corresponde a duas tabelas, tasks e task_events, para a troca por Postgres.